Name : Jigar Kumar Roll No. 23010303022 Section : B

Course : Cloud Computing (CS 3081) Mini Project

Github Repo : https://github.com/k-jigar/cloud-computing

Introduction :

In this mini project, I designed and implemented an end-to-end cloud-native system for a voice assistand command parser. The pipeline transfers capacity from a 3.8-billion parameter teacher model (Phi-3.5-mini instruct via Ollama) to a lightweight 70-million parameter student model (EleutherAI/pythia-70m) containerizes the inference service without baked weights, and deploys it on local kubernetes cluster (k3d) with S3 object storage (Seaweed FS), stateless NACL security, and automated CI/CD gating.

Learnings :

A. Sequence-Level VS. Logit-Based Distillation

Sequence-Level KD : $\mathcal{L}_{seq} = -\sum_{x=1}^{T} \log P_{student}(y_t | y_{<t}, x)$

Logit-Based KD : $\mathcal{L}_{logit} = (1-\alpha)\mathcal{L}_{CE} + \alpha T^2 D_{KL}(P^{(T)}_{teacher} \| P^{(T)}_{student})$

Logit-Based distillation requires identical vocabulary spaces. But output logit-dimension doesn't match ($\mathbb{R}^{32064} \neq \mathbb{R}^{50304}$), logit vectors cannot be aligned token-by-token, Sequence-Level was only feasible solution.

B. Storage Decoupling (12-Factor Cloud Principles)

Machine learning weights must never be baked into container images. Bundling it inflates container images, slowes down CI/CD builds.

C. Cloud-Native Orchestration & Networking

I learned how kubernetes provides process-level self-healing via liveness probes (/healthz) and readiness probes (/readyz). Stateless Network Access Control Lists (NACLs) operate at subnet boundaries without connection state tracking, meaning outbound response packets to client ephemeral ports (1024-65535) must be explicitly allowed.

Challenges Faced & Solutions Developed :-

1. Problem : Executing global Python binaries installed under user AppData triggered OS sandbox permission denials.
   Solution : Built a self-contained, isolated Python 3.12 virtual environment inside the workspace root (.venv) using uv.

2. Running parallel teacher labelling with Ollama alongside PyTorch training saturated all 6 physical CPU cores.
   Solution : I decoupled worker generation from student fine-tuning, capped active threads, and ran inference sequentially.

3. Problem : Despite strict system prompts, Phi-3.5 occasionally output conversational output wrong keys or hallucinated slot values not present in the command.
   Solution : Set Ollama options strictly : temperature = 0, seed = 42, num_predict = 55, with stop tokens ["\n\n", "Command :"].

4. Problem : When triggering a rolling update from model v1 to v2, old pods terminated before new pods had finished downloading model weights from S3, causing transition HTTP 503 drops.
   Solution : Tuned (cluster/app.yaml) readiness probe to query /readyz every 2 seconds with failure Threshold : 3. Kubernetes Traefik Ingress now routes traffic only after the model is resident in RAM, dropping failure rates to 0.2%.

Experimental Findings

Model Evalution Summary (100 validation Records)

Metric	Teachee (Phi-3.5, 3.8B)	Untrained Pythia	Distillied Student
JSON Validity	82.0%	0.0%	97.0%
Intent	74.0%	0%	62%
Data Accuray	66.0%	0%	52%
Time Accuray	74.0%	0%	60%
Place Accuray	68.0%	0%	55%
Strict Full Frame	42%	0	28%

Observation: Unfrained pythia-70M failed completely (0.0%)

Cloud Economics & FinOps Calculation :

Using unit rates from data/rate_card.json (1 vCPU : 3.50 Inr/hr, 1 GiB Memory : 0.45 INR/hr) :

1. Single Replica Hourly Cost :

   Cost = (1 vCPU x 3.50) + (1 GiB x 0.45) = 3.95 INR/hr.

2. Total Cluster Cost (2 Active Replicas) :

   Cost = 2 x 3.95 = 7.90 INR/hr.

3. Hourly Request Throughput : At measured locust load of 4.0 req/s.

   Request / hr = 4.0 x 3,600 = 14,400 req/hr.

4. Final Cost per 1000 Request :

$$\text{Cost per 1000} = \frac{\text{Hourly Cluster Cost}}{\text{Request per hour}} \times 1000 = \frac{7.90}{14,400} \times 1000$$

$$= 0.55 \text{ INR (55 paise)}$$